

NAME: S.GAYATHERI

REG NO:192521306

COURSE CODE: CSA0619

COURSE NAME : Design

Analysis of Algorithm

CO5 – AT1 --Constraint-Based Problem Solving

Q12 -Graph Isomorphism Verification

1. Problem Understanding

Given two graphs:

$$G_1 = (V_1, E_1)$$

and

$$G_2 = (V_2, E_2)$$

the objective is to determine whether they are isomorphic.

Two graphs are isomorphic if there exists a one-to-one mapping (bijection)

$$f: V_1 \rightarrow V_2$$

such that the adjacency relationship between every pair of vertices is preserved.

Formally,

$$(u, v) \in E_1 \iff (f(u), f(v)) \in E_2$$

If such a mapping exists, then:

$$G_1 \cong G_2$$

Otherwise, the graphs are not isomorphic.

2. Constraints for Vertex Mapping

The following constraints must be satisfied.

Constraint 1: Same Number of Vertices

$$|V_1| = |V_2|$$

If the number of vertices is different, the graphs cannot be isomorphic.

Constraint 2: Same Number of Edges

$$|E_1| = |E_2|$$

Isomorphic graphs must contain the same number of edges.

Constraint 3: One-to-One Vertex Mapping

Each vertex in G_1 must map to exactly one unique vertex in G_2 .

For example:

$$A \rightarrow 1, B \rightarrow 2, C \rightarrow 3$$

Two vertices cannot map to the same vertex.

Constraint 4: Degree Preservation

If a vertex has degree d in G_1 , it can only be mapped to a vertex with degree d in G_2 .

$$\deg(v) = \deg(f(v))$$

Constraint 5: Edge Preservation

If two vertices are connected in G_1 , their mapped vertices must also be connected in G_2 .

$$(u, v) \in E_1 \Rightarrow (f(u), f(v)) \in E_2$$

Similarly, non-edges should also be preserved.

3. Constraint Analysis

Without constraints, for n vertices there can be:

$$n!$$

possible vertex mappings.

For example, for 4 vertices:

$$4! = 24$$

possible mappings must be considered.

Constraint checking reduces unnecessary searching.

Important Pruning Conditions

Before assigning a vertex mapping, check:

1. Number of vertices is equal.
2. Number of edges is equal.
3. Degree sequences are equal.
4. The candidate vertex has not already been used.
5. The candidate vertices have compatible adjacency relationships.

Thus, invalid mappings are rejected early, reducing the search space.

4. Backtracking-Based Solution

The algorithm assigns vertices of G_1 to vertices of G_2 one at a time.

For each vertex:

1. Select an unmapped vertex from G_1 .
2. Try mapping it to an unused compatible vertex in G_2 .
3. Check degree and adjacency constraints.
4. If valid, continue with the next vertex.
5. If invalid, undo the assignment (backtracking).
6. If all vertices are successfully mapped, the graphs are isomorphic.

5. Algorithm

Step 1: Perform Initial Checks

Check whether:

$$|V_1| = |V_2|$$

and

$$|E_1| = |E_2|$$

Also compare their degree sequences.

If any condition fails, return Not Isomorphic.

Step 2: Start Vertex Mapping

Select an unmapped vertex from G_1 .

Step 3: Find Candidate Vertices

Consider unused vertices in G_2 having the same degree.

Step 4: Check Constraints

For every previously mapped vertex, verify that adjacency is preserved.

Step 5: Backtrack

If no candidate satisfies the constraints, undo the previous mapping and try another candidate.

Step 6: Final Decision

If all vertices are mapped successfully, return:

Graphs are Isomorphic

Otherwise:

Graphs are Not Isomorphic

Source code:

```
main.py
1  def is_isomorphic(graph1, graph2):
2      n = len(graph1)
3
4      # Constraint 1: Same number of vertices
5      if n != len(graph2):
6          return False
7
8      # Constraint 2: Same degree sequence
9      degree1 = sorted(len(graph1[i]) for i in range(n))
10     degree2 = sorted(len(graph2[i]) for i in range(n))
11
12     if degree1 != degree2:
13         return False
14
15     mapping = [-1] * n
16     used = [False] * n
17
18     def is_valid(v1, v2):
19         # Degree preservation
20         if len(graph1[v1]) != len(graph2[v2]):
21             return False
22
23         # Edge preservation with already mapped vertices
24         for u in range(n):
25             if mapping[u] != -1:
26                 edge1 = u in graph1[v1]
27                 edge2 = mapping[u] in graph2[v2]
28
29                 if edge1 != edge2:
30                     return False
31
32         return True
33
34     def backtrack(v):
35         # All vertices mapped successfully
36         if v == n:
37             return True
38
39         for candidate in range(n):
40
41             # Candidate must not already be used
42             if not used[candidate]:
43
44                 # Check constraints
45                 if is_valid(v, candidate):
46
47                     mapping[v] = candidate
48                     used[candidate] = True
49
50                     # Continue mapping
51                     if backtrack(v + 1):
52                         return True
53
54                     # Backtrack
55                     mapping[v] = -1
56                     used[candidate] = False
57
58         return False
59
60     return backtrack(0)
```

8. How Constraint Checking Eliminates Invalid Mappings

Suppose:

$$\deg(A) = 3$$

Then A cannot be mapped to a vertex with degree 2.

Similarly, if:

$$A \rightarrow 1$$

and A is connected to B , then the mapped vertices must satisfy:

$$1 \text{ is connected to } f(B)$$

If this condition fails, that mapping is immediately rejected.

Therefore, the algorithm does not explore all $n!$ mappings unnecessarily. It prunes invalid branches early using constraint checking.

9. Correctness Justification

The algorithm is correct because it considers every feasible one-to-one vertex mapping through backtracking.

- Every accepted mapping satisfies the degree constraint.
- No two vertices map to the same vertex.
- Every edge and non-edge relationship is checked.
- Therefore, a successful complete mapping is an isomorphism.

If no complete valid mapping exists, then no isomorphism exists.

Hence:

The algorithm returns True if and only if $G_1 \cong G_2$

10. Computational Classification

Graph Isomorphism is a decision problem:

Given two graphs, answer YES if they are isomorphic; otherwise answer NO.

The simple backtracking approach may examine up to:

$$n!$$

possible mappings.

Therefore, the worst-case time complexity is approximately:

$$O(n! \cdot n^2)$$

depending on the cost of constraint checking.

Space Complexity

The mapping and used arrays require:

$$O(n)$$

space, excluding the input graphs.

11. Conclusion

Graph isomorphism verification determines whether two graphs have the same structure despite different vertex labels. A constraint-based backtracking algorithm systematically searches for a valid vertex mapping while enforcing one-to-one mapping, degree preservation, and edge preservation. Early constraint checking eliminates invalid mappings and improves efficiency. Although basic backtracking can have factorial worst-case complexity, it guarantees correctness by exploring all feasible mappings.

Result: Constraint-based backtracking provides a correct method for deciding whether two graphs are structurally identical under vertex relabeling.